STOCKWHISK SAAS PLATFORM

DOC-06

# Multi-Tenant Data Isolation & Security Framework

Thread-Local Isolation Contexts, Scoped ORM Guards & Defense-in-Depth

| DOCUMENT TYPE | TARGET AUDIENCE | RELEASE VERSION | STATUS |
| --- | --- | --- | --- |
| Security Specification | Security Auditors, Backend Engineers | v1.4.0 Production | ✅ Verified & Approved |

## Multi-Tenant Security Requirements

StockWhisk is a multi-tenant SaaS application. Strict data isolation between shops (tenants) is the highest security priority.

### 1. Architecture

- **Thread-Local Context:** The application uses a thread-local storage mechanism ( `core.tenant_context` ) to store the current active tenant for the duration of a request.
- **TenantMiddleware:** Extracts the `Shop` context from the authenticated JWT user on every request and sets it in the thread-local context.
- **TenantScopedModel:** An abstract base Django model that includes a `shop` ForeignKey. Almost all business data models inherit from this.
- **TenantManager:** A custom ORM manager applied to `TenantScopedModel` s that automatically filters ALL queries by the current tenant. If no tenant is set in the context, it returns `qs.none()` to prevent accidental data leaks.

### 2. Data Isolation Requirements

- **SEC-01:** Every database query MUST be scoped to the current tenant. The `TenantManager` handles this automatically, but raw queries must be strictly reviewed.
- **SEC-02:** No cross-tenant data access is allowed, even if a user manipulates IDs (e.g., changing an invoice ID in a URL to one belonging to another shop). The `TenantManager` prevents this by enforcing the `shop_id` filter.
- **SEC-03:** File uploads (shop logos, document backups) MUST be stored in tenant-scoped directories (e.g., `/media/shop_{id}/` ).
- **SEC-04:** API responses MUST NEVER include relational data from other tenants. Serializers must use tenant-scoped querysets for relational fields.
- **SEC-05:** Bulk operations (creates, updates, deletes) MUST respect tenant boundaries and inject the correct `shop_id` .

## 3. Authentication & Authorization

- **Authentication:** JWT-based authentication with short-lived access tokens and longer-lived refresh tokens.
- **Authorization:** Role-Based Access Control (RBAC) per shop. A user's permissions are scoped strictly to their assigned role within a specific shop.
- **Platform Admin:** Platform administrators can impersonate tenants for support purposes. This MUST generate a severe audit log entry.
- **Demo Mode:** Read-only enforcement at the middleware/database level for demo accounts.
- **OTP Verification:** Phone number verification via OTP is required for registration. Password resets also utilize OTP.

## 4. API Security

- **Rate Limiting:** Enforced via DRF throttles ( `BurstUserThrottle` , `UserRateThrottle` ) to prevent brute force and DDoS attacks.
- **CORS:** Strictly configured to allow only trusted frontend domains.
- **HTTPS:** Enforced for all traffic; cookies are marked `Secure` and `HttpOnly` .
- **Input Validation:** Strict validation through DRF serializers. No direct mapping of request data to model fields without validation.
- **Injection Prevention:** Relying on Django ORM to prevent SQL injection. XSS prevention handled by React/Next.js escaping on the frontend and data sanitization on the backend if HTML is accepted.

## 5. Audit & Compliance

- **Financial Audit:** Append-only `LedgerEntry` system for all financial transactions. Modifications require reversing entries.
- **Inventory Audit:** Append-only `StockMovement` ledger.
- **Service Tracking:** `ServiceTicketStatusHistory` tracks every state change of a repair ticket.
- **User Tracking:** `last_seen` and IP tracking for active sessions.

## 6. Development Rules

- NEVER override `objects = models.Manager()` on a model that inherits from `TenantScopedModel` without explicitly re-implementing tenant filtering.
- ALWAYS use `TenantScopedModel` for new data models unless they are globally shared (e.g., Subscription Plans).